Postmortem PM-2026-014: Delivery outage in prod-us2 on 12 March 2026

Status: Complete. Severity: SEV1. Incident commander: Priya Raman. Author: Tomas Lindqvist.

Summary

On 12 March 2026, webhook deliveries in the prod-us2 region stalled for 47 minutes, from 14:06 to 14:53 UTC. Publishing kept working, so no events were lost, but 38% of deliveries in the region were delayed by more than ten minutes. Customers in prod-eu1 were not affected.

Impact

About 2,400 customers saw delayed deliveries. Four Enterprise customers opened support tickets. No data was lost and no event was delivered twice because of the incident. The Enterprise availability commitment covers the publishing API, which stayed up, so no service credits were owed.

Root cause

Version 3.5.0, deployed two days earlier, changed the retry scheduler to open a database transaction for every delivery it rescheduled and to keep it open while computing the backoff delay. Under normal load this was harmless. When a large customer endpoint started failing at 14:02, the number of deliveries waiting for a retry grew quickly and every one of them held a connection.

The connection pool reached its limit, db.pool.max_connections = 200. Delivery workers share that pool, so they could no longer record delivery attempts. Instead of failing quickly they waited for a free connection, because db.pool.acquire_timeout_ms was set to 30000. Workers spent most of their time blocked, and the delivery queue backed up across all partitions.

Timeline (UTC)

14:02 A large customer endpoint starts returning HTTP 503 for every request.
14:06 Delivery throughput in prod-us2 drops sharply. WL-ALERT-QLAG fires and pages the primary on-call engineer.
14:11 The primary scales up delivery workers. Lag keeps growing because the new workers also block on the pool.
14:19 SEV1 declared and an incident commander assigned. Status page updated at 14:24.
14:38 The team finds hundreds of idle-in-transaction connections owned by the retry scheduler.
14:44 The retry scheduler is paused. Connections are released and workers recover.
14:53 Delivery lag back to normal. Retry scheduler resumed with a hotfix.

What went well

Publishing was isolated from delivery, so customers could keep sending events. Pausing the retry scheduler was possible without a deploy.

What went poorly

Nothing alerted on connection pool saturation; we found it by reading database sessions by hand. Scaling up workers made the contention worse, which cost about 20 minutes.

Action items

ACT-112: Lower db.pool.acquire_timeout_ms from 30000 to 2000 so workers fail fast. Owner: Tomas Lindqvist. Done in 3.5.1.
ACT-113: Add an alert when database connections exceed 90% of the pool. Owner: Priya Raman. Done, see WL-ALERT-PGCONN.
ACT-114: Load test the retry scheduler with 100,000 pending retries before each release. Owner: Delivery team. Open.
ACT-115: Reschedule retries in batched updates instead of one transaction per delivery. Owner: Delivery team. Done in 3.5.1.